8주차 · 10.21 (수) · 중간고사 · 2026-2학기

08. 중간고사 서술형 대비 자료집 — 교안을 엮어서 풀어 쓰기

시험은 **서술형만** 나오고 **수식은 묻지 않는다**. 부분점수가 있으니 아는 것을 최대한 풀어 쓴다. 이 자료집은 교안 순서대로 문제를 세우고, 답은 교안과 필기의 표현을 살려 앞뒤 개념을 엮은 **문단형 서술**로 적었다. 실습(Python · OpenCV 코드) 내용은 뺐다.

오프라인 · 퇴계인문관 31705호 | 범위 1~7주차 | 서술형 · 부분점수 | 수식 ✕ · 개념 ○ | 실습 제외

자료집 PDF · 인쇄 · 오프라인용

컴퓨터비전 중간고사 서술형 대비 자료집

이 페이지 전체를 A4로 묶은 PDF다. 출력해서 문제만 보고 손으로 써 본 뒤 답과 비교하는 연습용으로 쓰면 된다.

GUIDE

시험 안내

출제 방식과 이 자료집의 답 쓰는 방식

일시 · 장소	8주차 **10월 21일(수)** · 오프라인 시험 · **퇴계인문관 31705호**
범위	1~7주차 교안 — Camera Models, Linear Filters, Image Pyramids, Local Feature Detection, Feature Descriptors, Feature Matching & Homography
형식	**서술형만** · **부분점수 있음** · 수식은 묻지 않는다 · 실습 제외
출제 예시	“RANSAC에 어떤 부분이 있는데, 그 부분은 어떻게 활용이 될까요?” — 알고리즘 속 한 부분을 짚고 그 쓰임을 묻는다

답 쓰는 방식 — 개념 하나를 따로 외우지 않고, 교안에서 **앞뒤로 이어지는 개념을 엮어** 한 흐름으로 쓴다. 예를 들어 핀홀 카메라를 물으면 구멍 크기의 딜레마 → 그래서 렌즈 → 렌즈의 대가인 초점 → 피사계 심도와 조리개 → 화각과 초점거리까지 이어 쓴다. 용어는 **영어 원어와 우리말을 함께** 적는다. 직접 정리 표시가 붙은 답은 내가 직접 적은 답이다.

W1

1주차 · 컴퓨터 비전이란

오리엔테이션

1-1

Q. 컴퓨터 비전이란 무엇인지 설명하시오.

답

[정의]

컴퓨터 비전은 인간 시각 시스템의 복잡한 처리 과정을 컴퓨터에 적용하여, 사람의 뇌가 하는 것과 같은 방식으로 영상 · 비디오 속 객체를 인식하고 처리하게 하는 컴퓨터과학 분야다. 대표 응용으로 image **classification**, localization, instance segmentation, object identification, object generation이 있다.

[인식 → 분류 → 이해]

컴퓨터 비전이 하려는 일은 인식(**recognition**) → 분류(**classification**) → 이해(**understanding**)로 이어진다. 그러기 위해 컴퓨터는 전체를 한 번에 보기보다 각각의 특징 · 요소들을 먼저 읽고, 조각조각 모아 가면서 전체를 이해한다. 필터로 **엣지**를 찾고, 다시 찾을 수 있는 특징점을 검출하고, 그 점을 벡터로 기술해 사진끼리 짝짓는 이번 학기의 흐름이 바로 이 '조각을 읽는' 과정이다.

2-1 직접 정리

Q. 이미지 형성의 네 요소에 대해 설명하시오.

답

[Geometry]

Geometry(기하학)은 3차원 세계에서 점과 점 간의 위치 관계를 설명하며, 컴퓨터비전에서의 카메라 모델이 이 위치 관계를 파악하기 위한 수단이 되기도 한다.

[Radiometry]

Radiometry(복사량)은 물리적인 빛의 양으로, 물체에 반사된 빛의 방향이나 양을 측정한다. 광학적으로 적외선이나 자외선까지 포함한다.

[Photometry]

Photometry(측광)는 사람이 눈으로 보는 빛의 양으로, 빛의 강도를 재는 것이다. **Radiometry**에서 사람 눈이 감지할 수 있는 범위만 다룬다.

[Digitization]

Digitization은 실세계의 아날로그 신호를 컴퓨터가 연산할 수 있는 이산적인 디지털 값으로 바꿔 주는 과정이다.

2-2 직접 정리

Q. 이미지 변환의 두 종류를 차이점 중심으로 설명하시오.

답

[두 갈래]

이미지 변환은 **Filtering**과 **Warping** 두 갈래가 있다.

[Filtering]

필터링은 이미지의 **화소값**을 건드는, 즉 빛의 양을 조절하는 변환이다. 화소 하나의 값을 정하기 위해 주변 화소를 전부 파악하며, 블러, **샤프닝**, **에지** 검출 등을 포함한다.

[Warping]

반면에 **와핑**은 형태를 건드는, 변환 함수를 활용해 다른 위치와 형태로 바꾸는 변환이다. 화소의 기하학적 관계를 파악해 옮기는 것이다.

2-3 직접 정리

Q. 선형 변환의 성질을 설명하시오.

답

[성질]

선형 변환은 원점에서 원점으로, 직선은 직선으로, 평행성은 유지하며, 길이의 비가 보존되는 변환이다. 변환의 합성은 곱의 형태로 표현된다.

[왜 곱셈인가]

대부분의 연산이 곱의 형태인 이유는, 곱셈 연산이 효율적이기 때문이다. 덧셈이 섞이면 변환을 이어 붙일 때마다 항이 늘어나고, 루프를 돌리면 어마어마한 다항식이 되어 버린다. 반면 곱셈 연산을 쓰면 변환을 시켜도 결국 곱셈으로 끝난다.

2-4 직접 정리

Q. 동차 좌표의 개념과 쓰이는 상황에 대해 설명하시오.

답

[개념과 쓰임]

동차 좌표는 **평행이동**을 덧셈이 아닌 곱의 표현으로 바꾸기 위해 좌표를 하나 더 붙이는 것이다. 동차 좌표는 컴퓨터비전에서 주로 **perspective 원근**에서 강력하게 쓰이며, 이동, 회전, 크기, 원근을 순서대로 곱해 하나의 행렬로 만들 수 있다.

2-5 직접 정리 + 보완

Q. 이미지 변환의 종류를 설명하시오.

답

[자유도 순서]

이미지 변환은 **자유도**(DoF)가 커지는 순서로 **translation**, **rigid**(**Euclidean**), **similarity**, **affine**, **projective**가 있다. 자유도는 몇 개가 바뀌느냐, 즉 내가 조절할 수 있는 손잡이의 개수다.

[평행이동 · 강체 · 유사]

Translation(**평행이동**)은 원래 영상을 새로운 위치로 옮기는 것으로, 이동량 두 개가 전부라 **자유도**는 2다. 모양 · 크기 · 방향이 그대로라 방향(orientation), 길이, 각도, 평행성, 직선을 모두 보존한다. Rigid(**Euclidean**, 강체) 변환은 이동에 θ만큼의 회전이 더해져 자유도 3이고, 길이를 보존한다. Similarity(유사 변환)는 여기에 크기 변화까지 더해 자유도 4이며, 길이는 바뀌지만 각도와 모양, 길이의 비를 보존한다.

[아핀 · 투영]

Affine 변환은 가로 · 세로가 모두 변환될 수 있어 매개변수가 6개다. **선형 변환**에 **평행이동**을 더한 것으로 원점과 매핑시킬 필요가 없고, 각도와 길이는 바뀌지만 평행성과 같은 직선 위 길이의 비는 보존한다. Projective(투영) 변환은 **원근**(**perspective**) 변환이자 **homography**로, 3D를 2D로 투영할 때나 두 이미지를 합성할 때(**파노라마**) 쓴다. 3×3 원소 중 전체 스케일은 의미가 없어 **자유도**는 8이며, 라인은 라인으로 가지만 평행도는 바뀌어 평행선이 **소실점**에서 만난다. 직선만 보존된다.

[포함 관계와 대응점]

위쪽 변환은 아래쪽 변환의 특수한 경우로 **translation** ⊂ **rigid** ⊂ **similarity** ⊂ **affine** ⊂ **projective**의 포함 관계가 있고, **자유도**가 2 → 3 → 4 → 6 → 8로 늘수록 보존되는 성질은 하나씩 사라진다. 대응점 한 쌍이 식 2개를 주므로 자유도는 곧 필요한 대응점 수가 되어, 아핀은 3쌍, 투영은 4쌍이면 변환 행렬을 구할 수 있다.

2-6

Q. 카메라 모델이 무엇을 설명하는지, 그리고 3D에서 2D로의 투영에서 무엇을 잃고 무엇을 지키는지 설명하시오.

답

[카메라 모델과 파라미터]

카메라 모델을 안다는 것은 이 카메라가 이미지를 획득했을 당시의 상황, 즉 카메라가 상황을 어떻게 받아들였고 어떻게 처리했는지를 이해한다는 뜻이다. 실제 카메라도 3차원에 있고, 그 위치에서 가져온 3차원 정보가 2차원으로 투영되는 과정을 수식으로 적는 것이 목표다. 카메라 파라미터에는 카메라를 어디에 두든 변하지 않는 **Intrinsic**(내부 요소, **초점거리** · **주점** · 픽셀 크기 · **skew**)과, 카메라를 옮기거나 돌리면 바뀌는 **Extrinsic**(외부 요소, 세계 좌표계에 대한 회전 R과 이동 t)이 있다. 반대로 사진들로부터 이 파라미터들을 역으로 추정하는 작업이 **Camera calibration**이다.

[투영의 성질]

투영은 3D를 2D로 줄이는 차원 축소 기계다. 같은 광선 위의 점들은 모두 같은 이미지 점으로 가는 **many-to-one**이라, 각도와 거리(길이), 깊이를 잃는다. 반면 직선과 선들의 교차점은 지킨다. 점은 점으로, 라인은 라인으로, 면은 면 혹은 반평면으로 간다(단, 광학 중심을 지나는 직선은 한 점이 된다). 또 하나의 가장 중요한 특징은 **vanishing point**(**소실점**)로, 평행했던 선들이 투영 후 소실점에서 만나 더 이상 평행하지 않게 된다. 방향마다 자기 소실점이 있고 같은 평면 위 소실점들을 이으면 **horizon line**(**소실선**)이 된다. 단, 이미지 평면과 평행한 직선은 투영 후에도 평행을 유지한다.

2-7 직접 정리 + 보완

Q. 핀홀 카메라 모델에 대해 설명하시오.

답

[핀홀 원리]

하나의 객체에서 나오는 여러 빛은 사방으로 퍼지기 때문에 필름의 여러 곳에 닿고, 아무 상도 맺히지 않는다. 이를 해결하기 위해 장벽을 설치하는데, 이 장벽을 통과한 하나의 빛이 필름에 맺힌다. 구멍에서 광선이 교차하므로 위는 아래로, 아래는 위로 뒤집혀 맺히는데 이 구멍 **핀홀**이 조리개가 된다. 계산 시에는 이미지 평면을 핀홀 앞쪽 위치로 옮겨 상이 위

아래로 바뀌지 않도록 세워 둔다.

[구멍 크기 딜레마]

핀홀 구멍이 크면 많은 빛이 들어와 이미지가 흐려지므로 가능한 작게 설정한다. 근데 또 너무 작으면 빛의 양이 적어 또 흐려질 수 있고, 빛이 가장자리에서 휘어 퍼지는 '**회절**(**diffraction**)' 현상까지 발생한다. 그래서 때에 따라 장시간 조리개를 열어 놓는 세팅을 하기도 한다.

> **[렌즈와 초점]**
> 이 딜레마를 푸는 것이 **렌즈**다. 빛이 들어올 때는 굴절(꺾임)의 문제가 있는데, 렌즈는 퍼지는 빛을 한 점으로 모아 주기 때문에 더 많은 빛으로 선명하게 표현할 수 있다. 평행하게 들어온 광선들이 모이는 점이 **초점**(focal point)이고, 렌즈에서 그 점까지의 거리가 **초점거리** f다. 렌즈 중심을 지나는 광선은 꺾이지 않으므로 **핀홀** 모델의 투영 관계는 그대로 쓸 수 있다. 대신 렌즈는 **thin lens formula**를 만족하는 거리, 즉 물체 거리 D와 상이 맺히는 거리 D′가 맞는 점만 정확히 초점이 맞는다. 렌즈의 모양 · 두께나 거리를 바꿔 이를 맞추는 것이 포커싱이고, 우리 눈도 같은 일을 한다(iris가 빛의 양을, retina가 센서 역할을 한다).

> **[착란원]**
> 그런데 그 거리에서 조금 벗어난 점도 **착란원**(**circle of confusion**)이 센서 화소보다 작으면 여전히 점으로 보인다.

[피사계 심도와 조리개]

잘 보이는 거리 구간을 **depth of field**라고 하고, 조리개 사이즈를 조절해서 이 구간을 조절한다. 조리개를 작게 조이면 **렌즈** 가장자리를 지나는 광선이 잘려 **circle of confusion**이 작아지고, 선명하게 커버하는 거리 범위가 길어진다. 대신 어두워진다. 반면 조리개를 크게 하면 빛이 많이 들어와 **착란원**이 커지고, 배경이 흐려지는 **아웃포커스**가 발생한다.

> **[조리개의 의미]**
> 결국 조리개는 **핀홀**에서의 구멍 크기 딜레마와 같은 '빛의 양 vs 선명함'의 맞바꿈을 **렌즈** 카메라에서 다시 조절하는 손잡이다.

> **[화각과 초점거리]**
> 한편 얼마나 넓게 담기는지는 **화각**(**Field of View**)이 정하며, 화각은 **초점거리** f와 센서 크기로 결정된다. f가 작아지면 더 넓은 각도가 유한한 이미지 평면에 들어와 **광각**(wider angle)이 되고, f가 커지면 화각이 좁아져 좁은 영역이 크게 확대되는 **망원**(telephoto)이 된다. 망원일수록 배경을 크롭시키는 효과가 있어 시점을 조금만 옮겨도 배경이 크게 바뀐다. 정리하면 **핀홀**은 기하(투영) 관계를, **렌즈**는 빛을 모으는 문제를, 조리개는 빛의 양과 **피사계 심도**를, 초점거리는 화각을 결정하며, 이 기하 관계를 행렬로 쓴 것이 **카메라 행렬 P**다.

2-8

Q. 카메라 행렬 P가 만들어지는 과정을 설명하시오.

답

[투영과 동차 좌표]

역상으로 맺힌 상을 당겨 이미지 평면을 optical axis(광학축) 위 **초점거리** f 위치에 놓으면(가상 이미지), 닮은꼴 삼각형에서 3D 점의 x, y를 깊이 z로 나누고 f를 곱한 위치에 상이 맺힌다. z로 나누는 연산 때문에 **선형 변환**이라고 할 수 없어 homogeneous하게 다루며, 그러면 전체가 x = PX라는 행렬 곱 하나가 된다.

[세 단계]

3D 점이 픽셀이 되기까지는 세 단계다. 먼저 world frame → camera frame으로, 실세계 좌표와 카메라의 이동성을 회전 R과 이동 t로 붙여 카메라 기준 좌표로 옮긴다(**Extrinsic**). 다음 camera frame → sensor plane으로, **초점거리**만큼의 배율을 도입해 투영한다. 마지막 sensor plane → image로, 센서 좌표의 원점은 광학축이 지나는 가운데지만 이미지 픽셀의 원점은 보통 왼쪽 위 모서리이므로 **주점**(**principal point**)만큼 **평행이동**을 더한다. 여기까지가 카메라 내부(**Intrinsic**)이고 이를 K로 묶는다.

[카메라 행렬 P]

동차 좌표를 쓰면 세 단계가 모두 행렬 곱이므로 미리 곱해 단 하나의 3×4 행렬 P = K[R | t]로 합쳐 둘 수 있고, 어떤 3D 점이든 곱셈 한 번으로 이미지 위치가 나온다. 더 일반적인 모델에서는 가로 · 세로를 다르게 scaling해야 할 수 있어(픽셀 종횡비) **자유도**가 증가하고, 축이 뒤틀려 있을 수도 있어 **skew** 보정값이 들어간다. 내부 5개와 외부 6개(R 3 + t 3)를 합한 11은 3×4 행렬 원소 12개에서 전체 스케일 1을 뺀 값과 같다. 이 파라미터를 알면 화면 속 위치를 실제 거리로 환산하거나 여러 대의 카메라 영상을 하나의 좌표계에서 이어 볼 수 있어, CCTV를 다루는 현장에서는 파라미터까지 조정한다.

2-9

Q. 카메라 영상에서 생기는 왜곡의 종류와 보정 방법을 설명하시오.

답

[원근 왜곡]

Perspective distortion(**원근** 왜곡)은 건축 사진의 고질적인 문제로, 높은 건물을 담으려고 카메라를 위로 기울이면 이미지 평면이 건물 벽과 평행하지 않게 되어 위는 좁게, 아래는 넓게 수직선들이 모인다(converging verticals). 카메라를 수평으로 유지하면 왜곡은 없지만 건물 아랫부분만 담긴다. 하드웨어로는 카메라는 수평인 채로 **렌즈**만 위로 **평행이동**시키는 **shifting lens**로, 우리가 배운 방법으로는 **호모그래피**로 왜곡된 평행선을 평행하게 바꿔 주는 **perspective rectification**으로 해결한다.

[방사 왜곡]

Radial distortion(방사 왜곡)은 어안**렌즈**처럼 가장자리가 왜곡되는 현상으로, 렌즈의 불완전함 때문에 가장자리를 지나는 광선일수록 편차가 크다. 가장자리가 밖으로 볼록하게 부푸는 **barrel**(술통)은 **광각** · 어안렌즈에서, 안으로 오목하게 당겨지는 **pin cushion**(바늘꽂이)은 **망원** 쪽에서 흔하다. 반지름 방향 왜곡 계수를 추정해 평행선을 유지하도록 당기고 늘리고 줄여서 맞추며, 결국 이것도 이미지에 대한 transformation이므로 **homography**와 함께 고려하고 카메라 파라미터를 활용해 보정한다.

3-1 직접 정리 + 다듬음

Q. 이미지와 필터링이란 무엇인가

답

[이미지와 컴퓨터비전]

이미지의 정의는 두 가지로 나눌 수 있다. 첫째, 현실 세계에 있는 것을 카메라라는 입력 장치를 통해 구현한 것이다. 둘째, 그래픽적 요소들을 포함해 상상으로 표현한 것이다. 컴퓨터비전은 이러한 영상 속 피사체를 인식(**recognition**)하고, 분류(**classification**)한 다음, 이해(**understanding**)한다. 이를 위해 전체를 한 번에 보기보다는 각각의 특징과 요소들을 먼저 읽고, 조각조각 모아 가면서 전체를 이해한다.

> **[디지털 이미지의 이산화]**
> 컴퓨터가 읽는 것은 **디지털 이미지**다. 빛을 통해 반사된 컬러의 값을 영상 센서가 인식해 화소로 이어 주므로, 그레이스케일 이미지는 좌표 (x, y)를 넣으면 그 위치의 밝기(intensity) 하나가 나오는, 2차원에서 1차원으로 가는 함수 f로 볼 수 있다. 이때 연속적인 장면은 두 번 이산화된다. 공간적인 측면(**spatial domain**)에서는 격자로 잘라 **샘플링**하고, 밝기 측면(**brightness domain**)에서는 0~255처럼 단계로 끊어 **양자화**한다. 그래서 이후의 모든 처리는 화소의 값을 연산해서 이해시키는 일이 된다.

[필터링]

이 조각을 읽기 위해, 기존 원 이미지에서 픽셀을 조합해 새로운 값의 픽셀을 만드는 것을 **필터링**이라 할 수 있다. 지난주의 **와핑**이 형태(좌표)를 건드렸다면, 필터링은 요소들은 그대로 두고 **화소값**, 즉 빛의 변화를 다룬다.

[노이즈와 필터링의 역할]

그런데 **디지털 이미지**에는 **노이즈**가 섞인다. 센서 퀄리티가 좋지 않거나, 빛이 균일하지 않게 반사되거나, 연속적인 밝기를 단계로 끊을 때 오차가 생기는(**양자화 효과**, quantization effects) 등 다양한 경우에 노이즈가 발생한다. 그래서 **필터링**이 던지는 질문은 '어떻게 노이즈를 줄일 것인가'와 '어떻게 유용한 특징을 찾을 것인가' 두 가지다. 필터링은 노이즈를 제거해서 피사체를 더 정확하게 인식하기 위해 활용되며, **에지**, **코너**, **블롭** 등 유용한 피처(feature)를 추출하기도 하고, **de-noising**(노이즈 제거), **super-resolution**(저해상도를 고해상도로), **in-painting**(복원) 등 영상의 속성을 향상 · 수정하기도 한다.

3-2

Q. 포인트 연산과 이웃 연산을 비교하고, 선형 시불변 필터가 무엇인지 설명하시오.

답

[포인트 연산 vs 이웃 연산]

포인트 연산(point operation)은 이미지의 크기 · 기하 · 국소 구조를 바꾸지 않고 화소 값만 매핑하는 것으로, 새 값은 같은 위치의 이전 값과 매핑 함수에만 의존한다. 그런데 하나의 화소를 가지고 처리하는 데는 한계가 있어 **샤프닝**이나 스무딩 같은 작업은 해낼 수 없고, 주변을 봐야 한다. 그래서 바꾸고자 하는 픽셀의 주변 요소들까지 고려하는 **이웃 연산**(neighborhood operation), 즉 필터가 필요하다.

[선형 시불변 필터]

선형 시불변(Linear Shift-Invariant) 필터는 각 픽셀을 이웃들의 선형 결합으로 바꾸며, 그 결합을 결정하는 것이 **커널**(kernel, 마스크)이다. 커널의 각 칸을 그 자리에 겹쳐지는 이미지 값과 곱해 모두 더한 값이 출력 한 칸이 된다(f = 입력 이미지, h = 커널, g = 출력 이미지). 같은 커널이 모든 픽셀 위치로 이동(shift)하므로 모든 픽셀이 같은 가중치 조합을 쓰는데, 이를 위치에 따라 변동이 없다는 뜻에서 **shift invariant**라고 한다.

3-3

Q. 평균 필터와 필터링 결과의 출력 크기에 대해 설명하시오.

답

[평균 필터]

평균 필터(square mean filter)는 각 픽셀을 주변의 값들을 평균 낸 국소 평균(local average)으로 바꾸며, 결과는 흐릿하게 변하는 스무딩(블러링) 효과다. 한 번에 필터가 움직이는 거리를 **stride**라 하고, 기본값 1칸에서 stride를 키우면 출력이 그만큼 더 작아진다.

[출력 크기]

출력 크기는 세 가지다. **Full**은 **커널**이 한 칸이라도 겹치는 모든 위치를 계산해 출력이 가장 크고(f와 h 크기의 합), **Same**은 **패딩**(pad)을 넣어 입력 f와 같은 크기로 맞추며, **Valid**는 커널이 완전히 이미지 안에 들어오는 위치만 계산해 크기가 줄어든다(f와 h 크기의 차). 예를 들어 10×10 이미지에 3×3 커널을 valid로 적용하면 8×8이 되고, same으로 두려면 한 칸씩 패딩을 넣는다.

3-4 질문 구성 직접

Q. 선형필터의 주요 성질을 설명하시오.

답

[선형성 Linearity]

두 필터를 더해서 적용한 결과는 각 필터를 따로 적용해 더한 결과와 같다. 즉 filter(h1 + h2) = filter(h1) + filter(h2)로 **선형성**이 보장된다. 출력이 이웃 화소의 가중합이기 때문에 성립하며, 덕분에 필터를 더하고 나누는 계산을 자유롭게 할 수 있다. 원본을 여러 배 하거나 블러를 빼는 식의 **커널** 조합이 가능한 것도 이 성질 덕분이다.

[시불변성 Shift invariance]

픽셀 위치와 무관하게 같은 동작을 한다. 이미지를 이동한 뒤 **필터링**한 것과 필터링한 뒤 이동한 것이 같다(filter(shift(h)) = shift(filter(h))). 화소가 움직이지 않는다는 뜻이 아니라, 같은 **커널**이 모든 위치로 옮겨 가며 어느 위치에서든 같은 가중치로 같게 동작한다는 의미다. 이 두 성질을 만족하는 모든 **선형 시불변** 연산자는 **컨볼루션**으로 표현

할 수 있다는 것이 이론적 결과다.

[커널에 따른 효과]

그래서 **커널**만 바꾸면 결과가 달라진다. 가운데만 1인 커널은 원본을 그대로 내놓고, 한쪽으로 치우친 1은 이미지를 한 칸 이동시키며, 모든 칸이 같은 값인 **박스 커널**은 국소 평균이 되어 흐려진다.

[샤프닝]

샤프닝 커널은 원본을 두 배로 키운 것에서 흐린 버전(박스 블러)을 뺀 것이다. 이는 원본 + (원본 − 흐린 것)과 같아서, 원본에 세부(디테일)를 한 번 더 얹는 셈이 된다. 원본 − 흐린 것은 블러가 지운 경계 · 질감, 즉 디테일만 남긴 것이므로, 이를 더하면 경계가 또렷해진다.

[언샤프 마스킹]

이것이 **언샤프 마스킹**(**unsharp masking**)의 원리다. 흐린, 즉 'unsharp'한 버전을 마스크로 만들어 원본에서 빼서 디테일을 얻고, 그 디테일을 다시 원본에 더해 선명하게 만든다는 뜻에서 붙은 이름이다. 이름과 달리 결과는 **샤프닝**이며, **선형성** 덕분에 이 과정 전체(원본 두 배 − 블러)를 **커널** 하나로 합쳐 한 번에 적용할 수 있다.

3-5 질문 구성 직접

Q. 마스크 크기는 어느 정도로 설정해야 하는가?

답

[마스크가 커지면]

마스크(mask)는 **커널**과 같은 말로, 보통 3×3, 5×5, 7×7을 사용한다. 마스크가 커지면 가져오는 주변 화소들이 많아져 더 많은 이웃이 결과에 기여한다. 평균 내는 표본이 많아지므로 출력의 **노이즈** 분산은 감소해 결과가 안정되지만, 대신 한 점의 노이즈가 더 넓게 번지고(bigger noise spread), 이미지가 더 흐릿해지며(more blurring), 마스크가 클수록 연산이 더 부담이 되는 문제가 발생한다. 즉 크게 하면 노이즈는 잘 줄지만 디테일과 속도를 잃고, 작게 하면 디테일은 살지만 노이즈가 덜 줄어드는 맞바꿈이 있어 정답인 크기는 없다.

[박스 필터의 격자 문제]

또 **평균 필터**로 블러링하면 격자가 살짝 보이는 문제가 생길 수 있다. 네모 영역에 해당하는 값들을 가져오다 보니 네모 블러가 생기는 것인데, **박스 필터**가 중심에서 멀든 가깝든 창 안의 모든 화소를 거리와 무관하게 똑같은 가중치로 평균 내기 때문이다(박스 필터는 정사각형 창 안의 화소를 모두 같은 가중치로 더해 평균 내는 평균 필터를 의미한다). 중심에서의 거리를 반영하지 않으므로 정사각형 창의 모양이 결과에 그대로 새겨지고, 마스크가 클수록 이 자국이 더 뚜렷해진다.

[그래서]

그래서 마스크는 **노이즈** 제거와 디테일 · 연산량 사이에서 목적에 맞게 크기를 정하되, 단순히 키우기보다 중심에 가까운 정도에 따라 가중치를 줘야 한다. 이것이 센터에는 가중치를 많이 두고 멀어질수록 줄이는 **가우시안 필터**이며, **가우시안**은 표준편차 σ로 활용 거리를 통제하므로 마스크 크기도 σ에 맞춰(가중치가 의미 있는 범위만큼) 정하면 된다.

3-6

Q. 가우시안 필터의 특징과 장점을 설명하시오.

답

[가우시안 필터]

가우시안 필터는 **평균 필터**의 특수한 경우로, 계수가 2D **가우시안**의 표본이다. 센터로부터 원형 형태를 유지하면서 최대한 부드럽게 만드는데, 센터에는 가중치를 많이 두고 가장자리로 갈수록 weight를 줄인다. 표준편차 σ를 이용해서 거리를 통제하며, 표준편차가 클수록 활용 거리가 넓어져 더 흐려진다.

[장점]

장점은 다음과 같다. 큰 필터에서도 어떤 방향이든 상관없이 symmetric한 **회전 대칭**이라 **박스 필터**의 네모 자국이 사라진다. 중심 봉우리에서 멀어질수록 가중치가 **단조 감소**해 중심 화소에 가장 큰 비중을 준다. 시그마가 얼마냐에 따라 흐림 정도가 정해져 간단하고 직관적이다. **가우시안**끼리 **컨볼루션**하면 또 다른 가우시안이 되어(**self-convolution**), σ1과 σ2를 잇달아 적용한 것이 한 번의 더 큰 가우시안과 같다.

[분리 가능성]

또 **분리 가능**(**separability**)하다. 2D **가우시안**은 x 방향 가우시안 × y 방향 가우시안으로 정확히 쪼개지므로, k×k 필터를 2D로 하면 픽셀당 k^2번이던 연산이 1D 필터 두 번으로 2k번으로 줄어든다. 이 가우시안은 이후 축소 전 블러(4주차), **Canny**와 **LoG**(5주차), **SIFT**의 **스케일 공간**(6주차)에 계속 쓰인다.

3-7

Q. 비선형 필터링, 특히 미디언 필터에 대해 설명하시오.

답

[노이즈와 선형 필터의 한계]

노이즈에는 검은 점과 흰 점이 무작위로 나타나는 **Salt and pepper**, 흰 점이 무작위로 나타나는(peak noise) **Impulse**, 밝기 변동이 정규분포에서 뽑히는 Gaussian이 있다. 이런 노이즈가 있을 때 선형 필터를 쓰면, 평균이 튀는 값(**outlier**)을 그대로 반영하기 때문에 점 하나의 극단적인 값이 이웃 전체로 번져 나가 노이즈 영향이 퍼진다. 그래서 **비선형 필터링**으로 노이즈를 제거한다.

[미디언 필터]

미디언 필터는 각 픽셀을 이웃 화소들의 **중앙값**(median)으로 바꾼다. 가중합으로 쓸 수 없는 **비선형** 필터이며, **노이즈**를 퍼뜨리지 않고(does not spread the noise) 스파이크 노이즈를 제거할 수 있어 임펄스, 소금 · 후추 노이즈에 좋다. Mean 필터에 비해 **outlier**에 덜 민감하다. 단점은 정렬이 필요해 실행 비용이 비싸다는 것이다.

3-8

Q. Convolution과 Cross-correlation을 비교하고, 컨볼루션의 성질을 설명하시오.

답

[컨볼루션 vs 상관]

컨볼루션은 필터를 두 방향 모두 뒤집은 뒤(위가 아래로, 좌가 우로, flip the filter in both dimensions) **cross-correlation**을 적용하는 것이다. 개념적으로 컨볼루션은 한 함수가 다른 함수 위를 이동할 때 발생하는 겹침의 양을 표현하는 **필터링** 연산으로, 필터를 거친 결과(시스템 응답)를 계산한다. 반면 cross-correlation은 뒤집기 없이 원형 그대로 겹쳐 두 신호가 이동하면서 나타나는 유사도(measure of **similarity**)를 계산하며, 두 신호가 가장 잘 일치하는 지점에서 최댓값에 도달한다. 박스나 **가우시안**처럼 **커널**이 대칭이면 뒤집어도 같아 두 연산의 결과가 같다.

[컨볼루션의 성질]

컨볼루션은 **교환법칙**, **결합법칙**, **분배법칙**, 스칼라곱과의 결합, 미분 성질을 가진다. 특히 결합법칙 덕분에 두 필터를 미리 하나로 합쳐 한 번만 돌릴 수 있어 **분리 가능**성의 근거가 되고, 미분 성질 덕분에 이미지를 미분하는 대신 필터 쪽에 미리 미분을 적용할 수 있어 **가우시안**의 미분(**DoG**) 필터가 가능해진다.

3-9

Q. 에지 검출에 대해 설명하시오. (Prewitt, Sobel, DoG, LoG)

답

[에지와 스무딩]

에지들은 물체의 형상을 설명한다. surface(표면), depth(깊이), surface color(표면 색), illumination(조명)의 discontinuity가 모두 에지로 나타난다. 에지는 밝기의 변화이므로 미분으로 찾는데, 미분은 **노이즈**까지 키우기 때문에 노이즈 필터를 적용하고 에지 감지를 적용한다. 스무딩과 미분이 둘 다 **컨볼루션**이고 **결합법칙**을 만족하므로 두 마스크를 미리 하나로 합쳐 한 번에 돌릴 수 있다.

[Prewitt · Sobel]

Prewitt 에지 검출기는 vertical과 horizontal을 나눠서, 한 축으로는 [1 1 1] 스무딩이, 다른 축으로는 [−1 0 1] 미분이 동시에 일어나 **노이즈** 제거와 에지 검출을 한 마스크에 담았다. **Sobel**은 가중치를 더 살려 [1 2 1]로 가운데를 더 믿으므로 **가우시안**에 가까운 부드러운 스무딩이 되어 노이즈에 조금 더 강하다.

[DoG · LoG]

DoG(**Derivative of Gaussian**)는 **가우시안**의 1차 미분이다. 미분도 **컨볼루션**이고 **결합법칙**을 만족하므로, 스무딩한 뒤 미분하는 대신 가우시안을 미리 미분해 둔 필터를 한 번만 적용해 연산을 아낀다. **LoG**(**Laplacian of Gaussian**)는 가우시안의 2차 미분으로, 1차 미분의 극값 대신 2차 미분의 **영교차**(**zero-crossing**)로 **에지**를 찾는다. 2차 미분은 에지 양쪽에 부호가 반대인 봉우리를 만들어 선이 이중으로 보이는 현상이 있고, 그 사이 0을 지나는 지점이 진짜 에지다. 또 나눠서 할 수가 없어(분리 불가) 연산량이 많다. 이 교안의 DoG는 Derivative of Gaussian이지만, 5 · 6주차의 **블롭** 검출과 **SIFT**에서는 DoG가 **Difference of Gaussians**를 뜻하므로 구분해야 한다.

4주차 · Image Pyramids

Aliasing → Gaussian · Laplacian · Wavelet → Interpolation

4-1

Q. 다운샘플링 시 발생하는 에일리어싱과 그 해결 방법을 설명하시오.

답

[다운샘플링과 에일리어싱]

다운샘플링은 이미지의 해상도를 줄이는 것으로, 가장 단순하게는 짝수 번째 행과 열만 남기고 나머지를 버린다. 데이터와 연산량을 줄이고, 세부를 걷어내 큰 구조만 보고, 피라미드를 만들기 위해 한다. 그런데 그냥 버리면 **에일리어싱**(**aliasing**)이 생긴다. 에일리어싱은 연속 신호를 띄엄띄엄 찍어 디지털로 바꾸는 과정에서 샘플이 충분히 촘촘하지 않아 원래와 다른 신호로 둔갑하는 왜곡이다. 가는 줄무늬나 경계가 계단 모양으로 깨지고 원래 없던 **모아레**가 나타나는데, 단순히 흐려지는 게 아니라 틀린 무늬가 새로 생긴다는 점이 중요하다. 바퀴가 멈추거나 거꾸로 도는 듯 보이는 마차 바퀴 효과는 시간축에서의 에일리어싱이다.

[원인과 해결]

원인은 **샘플링** 정리(**나이퀴스트**)로 설명된다. 신호가 한 번 오르내리는 동안 최소 두 번은 찍어야 하므로, 샘플링 주파수가 신호의 최대 주파수의 2배 이상이어야 한다. 이보다 성기면 샘플링으로 복제된 스펙트럼끼리 겹쳐 원래 신호를 되찾을 수 없다. 피하는 방법은 두 가지로, 더 촘촘히 찍거나(**오버샘플링**), 이미 찍힌 이미지라면 **가우시안** 블러로 깨질 고주파를 먼저 지운 뒤 줄이는 것이다. 순서가 핵심으로, 줄이고 나서 블러하면 이미 생긴 가짜 무늬가 흐려질 뿐이므로 반드시 블러 → 축소여야 한다. 이 '블러 후 축소'를 반복한 것이 **가우시안 피라미드**다.

4-2

Q. 가우시안 피라미드에 대해 설명하시오.

답

[구성]

가우시안 피라미드는 한 장의 이미지에 "블러 → 절반으로 줄이기"를 반복해 여러 해상도 버전을 쌓아 둔 것이다. 원본을 맨 아래(Level 0)에 두고 작아진 것을 위로 쌓으면 피라미드 모양이 된다. 한 층 올라갈 때마다 해상도는 1/2, 픽셀 수는 1/4이 되어 전부 더해도 원본의 약 4/3배밖에 되지 않는다. 위로 갈수록 가는 무늬 · 질감 · 작은 물체가 지워지고 크고 균일한 영역(uniform regions)만 남아, 나무를 지우고 숲만 남긴 이미지가 된다. 그래서 위층에서는 전체 배치와 큰 구조를 빠르게 파악하고, 아래층에서는 세밀한 정보를 얻는다.

[활용과 한계]

이렇게 크기를 바꿔 가며 보는 도구는 검출기의 창 크기가 고정이어도 이미지를 줄여 모든 크기의 물체를 잡는 **스케일 불변** 검출(**SIFT**의 **스케일 공간**), 작은 층에서 대략의 답을 구하고 다음 층에서 보정하는 **coarse-to-fine**(옵티컬 플로우, 정합), 연산량 절감에 쓰인다. 대신 블러로 지운 고주파는 어디에도 남지 않아 위층만으로는 원본을 되살릴 수 없다. "잃어버린 것을 따로 보관하면 되지 않나"라는 물음에서 **라플라시안 피라미드**가 나온다.

4-3

Q. 라플라시안 피라미드에 대해 설명하시오.

답

[구성과 복원]

한 층 위 이미지를 다시 키운 것은 원래 층의 '예측'이라고 볼 수 있다. 블러로 지워진 디테일이 없으니 원래 층과 조금 다른데, **라플라시안 피라미드**는 블러 이미지 대신 이 차이(예측 오차, **잔차**)를 저장한다. 블록도로 보면 Smooth · Down-sampling이 **가우시안 피라미드**를 한 층 만드는 부분이고, 그 위층을 다시 Up-sampling · Interpolate해 만든 예측을 원래 층에서 뺀 Prediction error가 라플라시안 층이다. 저장하는 것은 맨 위의 작은 이미지 한 장과 층마다의 잔차이고, 위에서부터 '키우고 더하기'를 반복하면 원본이 정확히 돌아온다. **가우시안**만으로는 불가능했던 복원이 잔차 덕분에 가능해진다.

[이름과 활용]

잔차 이미지는 대부분 0 근처라 회색빛에 윤곽선만 떠 있고 히스토그램이 가운데 한 점에 뾰족하게 몰린다. 원본 – 블러는 흐림 정도가 다른 두 **가우시안**의 차이이고 이는 **LoG**의 근사이므로, 잔차가 **엣지** 검출 결과처럼 보여 '라플라시안'이라는 이름이 붙었다. 이 성질로 0에 몰린 값을 적은 비트에 담는 **압축**(Burt & Adelson의 원래 목적)에 쓰고, 두 이미지를 층마다 따로 섞어 저주파(밝기 · 색)는 넓게, 고주파(엣지 · 질감)는 좁게 섞는 이미지 **블렌딩**(사과 + 오렌지, 7주차 **파노라마**의 **다중 대역 블렌딩**)에 쓴다.

4-4

Q. 웨이블릿 변환에 대해 설명하시오.

답

[분해 방식]

웨이블릿 부호화는 DCT 부호화의 변형으로, DCT처럼 블록으로 자르지 않고 웨이블릿을 쓴다(JPEG 2000). 한 번 쪼개면 저주파는 원본을 절반 크기로 줄인 것 같은 근사(approximation, LL)로 가고, 고주파는 방향에 따라 가로(LH) · 세로(HL) · 대각(HH) 셋으로 나뉜다. 그리고 근사 쪽만 같은 과정을 반복한다(Level 2, 3 …). 계수는 모두 합쳐도 원본과 똑같이 N개라서, 4/3배로 늘어나는 **라플라시안 피라미드**와 달리 정보를 잃지도 늘리지도 않는다.

[근사만 쪼개는 이유]

근사 쪽만 다시 쪼개는 이유는 자연 영상의 에너지가 대부분 저주파에 몰려 있고, 디테일은 이미 0에 가까운 반면 근사는 큰 값이 빽빽해 **압축**이 어렵기 때문이다. 저주파를 계속 털어내 0에 가까운 계수를 대량으로 얻는 것이다. 다만 고주파 계수를 많이 버릴수록 경계 주변에 잔물결(**링잉**)이 생긴다.

4-5

Q. 보간(Interpolation)에 대해 설명하시오.

답

[보간과 재구성 필터]

보간은 이미지를 키울 때 생기는 빈칸을 채우는 일이다. 원래의 연속적인 장면 f를 알면 어떤 해상도로든 다시 만들 수 있지만 우리에게는 찍힌 픽셀 값뿐이므로, f를 근사하는 **재구성 필터** h를 정해서 채운다. 샘플 사이를 일단 0으로 채운 뒤 재구성 필터를 **컨볼루션**해 매끄럽게 잇는 것으로, 어떤 필터를 쓰느냐가 곧 어떤 보간이냐를 결정한다. 이론상 이상적인 **sinc**는 좌우로 무한히 출렁여 실제로 쓸 수 없고 **링잉**이 생긴다.

[세 가지 보간]

최근접 이웃(**박스 필터**)은 가장 가까운 값을 그대로 복사해 가장 빠르지만 계단(블록) 현상이 심하다. **바이리니어**(삼각형 필터)는 주변 네 점을 써서 x로 두 번, y로 한 번 섞으며, 가까운 쪽 값에 더 큰 가중치가 붙는다. 계단은 사라지지만 경계가 뿌옇게 흐려진다. **바이큐빅**은 4×4 = 16점을 3차 곡선으로 이어 기울기까지 매끄럽고, 음수 가중치가 경계를 선명하게 살리지만 계산량이 크고 오버슈트가 생길 수 있다. 이 **보간**은 피라미드에서 위층을 키울 때와, 7주차 **와핑**에서 변환된 소수점 좌표의 값을 가져올 때도 쓰인다.

W5

5주차 · Local Feature Detection

Local Features → Edge → Line(Hough) → Corner(Harris) → Blob(LoG · DoG)

5-1

Q. 로컬 특징이 무엇이며 왜 필요한지 설명하시오.

답

[필요성]

파노라마를 만들려면 두 사진에서 같은 지점을 가리키는 짝을 찾아야 하는데, 픽셀 전체를 하나하나 비교하면 느리고 밝기가 조금만 달라져도 실패한다. 그래서 사진마다 눈에 띄는 지점을 뽑고(추출) → 지점끼리 짝을 짓고(매칭) → 그 짝으로 사진을 정렬한다. 이미지 매칭, 파노라마, 3차원 복원, 모션 추적(AR), 물체 인식, 이미지 검색(retrieval), 로봇 내비게이션이 모두 이 도구 위에 서 있다.

[장점 · 단계 · 종류]

로컬 특징은 작은 영역에만 의존해 가림(occlusion)이나 복잡한 배경(clutter)에도 나머지 특징이 살아남는 **Locality**, 한 장에서 수백~수천 개가 나와 몇 개가 틀려도 버티는 **Quantity**, 대규모 데이터베이스에서도 구별되는 **Distinctiveness**, 실시간 처리가 가능한 **Efficiency**의 장점이 있다. 다루는 과정은 관심점이 어디 있는지 찾는 **Detection**(Find feature, 5주차), 주변을 벡터로 요약하는 **Description**(Represent feature, 6주차), 두 사진의 **기술자**를 비교해 짝짓는 **Matching**(Match feature, 7주차)의 세 단계다. 특징의 종류는 밝기가 급격히 변하는 **엣지**, 엣지 점들이 일직선으로 늘어선 직선, 어느 방향으로 움직여도 밝기가 크게 변하는 **코너**, 주변과 구별되는 덩어리인 **블롭**이 있으며, 점 → 선 → 모서리 → 덩어리로 갈수록 '다른 사진에서도 다시 찾을 수 있는 좋은 특징'에 가까워진다.

5-2

Q. 엣지 검출의 원리와 과정을 설명하시오.

답

[원리]

엣지는 이웃한 두 점 사이의 밝기 변화량이 큰 곳이고, 변화량을 재는 수학 도구가 미분이므로 엣지는 1차 미분의 극값이다. 디지털 영상은 정수 위치에만 픽셀이 있어 옮길 수 있는 최소 거리가 한 칸이므로, 미분은 '옆 픽셀 – 지금 픽셀'의 **유한 차분**이 되고 이 빼기를 작은 표로 적은 것이 **커널** 마스크다. 이미지는 2차원이라 x, y로 각각 **편미분**하는데, x 방향 미분은 가로로 가다 부딪히는 세로 엣지를 찾는다. 두 편미분을 묶은 **그래디언트**는 밝기가 가장 빠르게 증가하는 방향을 가리키며, 그 크기(magnitude)가 엣지의 세기이고 방향은 엣지를 가로지르는 방향이다. 나침반 8방향 마스크를 모두 걸어 최대 응답으로 크기와 방향을 정하는 컴퍼스 그래디언트 마스크도 있다.

[과정과 연산자]

실제 사진에는 **노이즈**가 있어 그냥 미분하면 진짜 경계와 잡음이 뒤섞인다. 그래서 스무딩 · 블러링으로 잡음을 누르는 **Filtering** → 차분 연산자로 변화가 큰 픽셀을 도드라지게 하는 **Enhancement** → 임계값을 넘는 강한 **엣지**만 남기는 **Detection**의 3단계를 거친다. 블러를 세게 할수록 잡음은 잘 지워지지만 엣지도 무뎌져, edge strength와 noise

reduction 사이의 **tradeoff**가 있다. 차분 연산자로는 세 줄의 차분을 같은 무게로 평균하는 **Prewitt**, 가운데 줄에 가중치 2를 둬 잡음에 더 강한 **Sobel**, 2×2 대각선 차분으로 가장 빠르지만 잡음에 취약한 **Roberts**가 있고, 임계값은 연산자마다(Roberts만 40) 상황에 따라 조정한다.

5-3

Q. Canny 엣지 검출기에 대해 설명하시오.

답

[문제와 앞 단계]

Canny는 단순한 **그래디언트 엣지**의 세 가지 문제, 즉 스케일마다 그래디언트 크기가 달라 어느 σ를 골라야 하는지, 그래디언트가 큰 영역이 두꺼운 띠로 나오는데 어느 점이 진짜 엣지인지, 남은 점들을 어떻게 이어 곡선으로 만드는지를 풀려는 4단계 절차다. ① **가우시안** 미분으로 **필터링**해 스무딩과 미분을 한 번에 하고, ② 각 픽셀의 그래디언트 크기와 방향을 구한다. 방향은 다음 단계에서 비교 상대를 고르는 데 쓰인다.

[NMS와 이력 임계값]

③ **비최대 억제**(**Non-maximum suppression**)는 두꺼운 능선을 한 픽셀 폭으로 깎는다. 현재 픽셀 q를 **그래디언트** 방향으로 앞뒤에 있는 p, r과 비교해 둘 다보다 크면 꼭대기이므로 남기고 아니면 0으로 지운다. 비교는 **엣지**를 따라서가 아니라 가로질러서 하는데, 엣지를 따라 비교하면 같은 선 위의 이웃끼리 지워 선이 끊기기 때문이다. ④ **이력 임계값**(**Hysteresis thresholding**)은 임계값을 두 개 쓴다. high 이상은 강한 엣지로 무조건 채택해 곡선을 시작하고(use a high threshold to start), low 미만은 버리며, 그 사이의 약한 엣지는 강한 엣지와 이어져 있을 때만 살린다(a low threshold to continue). 높은 임계값만 쓰면 깨끗하지만 끊기고, 낮은 임계값만 쓰면 잡음까지 살아나는데, 진짜 경계는 길게 이어지고 잡음은 뚝 떨어진 점으로 나타나므로 연결성으로 둘을 가려 strong의 깨끗함과 weak의 연결성을 함께 얻는다.

5-4

Q. 허프 변환에 대해 설명하시오.

답

[투표 아이디어]

허프 변환(**Hough Transform**)은 이미지 안에서 특정 모양을 분리해 내는 기법으로, **엣지** 검출 후 흩뿌려진 엣지 점들 중 일직선 위에 놓인 점들을 찾는다. 점 두 개씩 짝지어 직선을 확인하는 대신, 점들이 자기가 속할 수 있는 직선에 투표하게 한다. 직선 y = ax + b는 (a, b) 두 숫자로 정해지므로 **파라미터 공간**의 점 하나가 이미지의 직선 하나이고, 반대로 이미지의 점 하나는 그 점을 지나는 직선들의 목록이라 파라미터 공간의 직선 하나가 된다. 같은 직선 위의 점들이 만든 직선들은 한 점에서 교차하고, 그 교점이 찾는 직선이다.

[누적 배열과 (ρ, θ)]

컴퓨터는 **파라미터 공간**을 칸으로 나눈 배열(**누적 배열**, **accumulator**)을 만들고, a_max · b_max로 범위를 정한 뒤, **엣지** 점마다 지나는 칸에 투표해 가장 높은 (a, b)를 고른다. 잡음 점의 표는 여기저기 흩어져 결과를 흔들지 못하고, 점 몇 개가 빠져도 나머지 표로 1등이 되므로 잡음과 끊김에 강하다. 그런데 (a, b)는 세로선의 기울기가 무한대라 적을 수 없어, 원점에서 직선에 내린 수직선의 길이 ρ와 각도 θ로 표현한다(avoid infinite slope). θ와 ρ의 범위가 유한해 누적 배열이 항상 유한하며, 이때 점 하나는 사인 곡선이 되고 **국소 최대**(local maxima)가 뚜렷한 직선이 된다. 다만 칸을 너무 잘게 나누면 표가 흩어지고 너무 굵게 나누면 다른 직선이 뭉치는 해상도 문제가 있다.

5-5

Q. 코너의 개념과 Harris 코너 검출기에 대해 설명하시오.

답

[코너란]

좋은 특징은 다른 사진에서도 헷갈리지 않고 짝을 찾을 수 있는(unambiguous) 지점이다. **코너** 자체는 하나의 점이지만 점 하나만 봐서는 알 수 없어, 점 주변에 작은 창(window)을 씌우고 이리저리 밀어 본다. 평탄(flat)한 곳은 어느 방향으로 밀어도 변화가 없고, **엣지**(edge)는 엣지 방향으로는 변화가 없고 가로지르는 방향으로만 변해 선을 따라 미끄러지며(**구멍 문제**, **aperture problem**), 코너(corner)는 모든 방향으로 크게 변해 위치가 한 점으로 고정된다.

[Harris]

Harris는 창을 밀었을 때의 변화량을 **SSD**로 적는데, 모든 픽셀마다 모든 이동량을 계산하기엔 너무 느리다. 그래서 이동이 작다는 가정으로 테일러 근사를 하면 변화량이 **그래디언트**로 만든 2×2 행렬, 즉 **2차 모멘트 행렬**(**second moment matrix**)로 표현된다. 창 안 그래디언트들의 분포를 담은 이 행렬을 한 번의 **필터링**으로 구해 두면 모든 방향의 변화를 다시 계산할 필요가 없다. 행렬의 **고윳값**은 두 방향으로 밀었을 때의 변화 크기이고 고유벡터는 그 방향이라, 둘 다 작으면 평탄, 하나만 크면 **엣지**, 둘 다 크고 비슷하면 **코너**다. 고윳값을 직접 구하지 않으려고 **행렬식**과 대각합으로 **응답 함수** R(Harris & Stephens)을 만들고, Shi & Tomasi는 작은 고윳값을 점수로 쓴다. 정리하면 그래디언트 계산 → H 행렬 구성 → 고윳값 계산 → 임계값 적용의 네 단계이며, 마지막에 **국소 최대**만 남겨 한 코너에서 여러 점이 중복 검출되는 것을 막는다.

5-6

Q. Harris 코너 검출기의 불변성에 대해 설명하시오.

답

[강한 것]

밝기 변화에서, 전체가 밝아지는 덧셈은 Harris가 미분만 쓰므로 미분에서 사라져 완전히 불변이다. 대비가 커지는 곱셈은 R 값 자체는 커지지만 어느 점이 더 큰가의 순서는 유지되므로 부분적으로만 불변(partially **invariant** to **affine** intensity change)이다. 이동은 창이 같이 움직여 **코너** 위치도 같이 따라 움직이는 공변이고, 회전은 행렬의 타원이 이미지와 함께 돌지만 모양(**고윳값**)은 그대로라 같은 코너가 다시 검출된다.

[약한 것 — 스케일]

그런데 스케일에서 무너진다. 같은 물체를 확대해서 찍으면 창 크기는 그대로인데 무늬만 커져, 뾰족했던 모서리가 창 안에서 완만한 곡선으로 보이고 모든 점이 **엣지**로 분류된다. 해결 방향은 위치와 스케일 양쪽에서 **응답 함수**가 **국소 최대**가 되는 스케일을 찾는 것이고, 창을 키우는 대신 이미지를 줄여 가며(4주차 **가우시안 피라미드**) 같은 창으로 훑으면 훨씬 싸게 같은 효과를 얻는다. 이 생각이 **블롭** 검출로 이어진다.

5-7

Q. 블롭 검출에 대해 설명하시오. (LoG, 스케일 정규화, DoG)

답

[LoG]

블롭은 주변과 밝기가 다른 덩어리 영역으로, 위치뿐 아니라 크기까지 찾아야 한다. **LoG**(**Laplacian of Gaussian**)는 **가우시안**으로 부드럽게 만든 뒤 2차 미분(라플라시안)을 거는 연산자로, 단면이 멕시코 모자 모양이라 블롭처럼 생겼고 원형 대칭이다. 2차 미분은 잡음에 민감해 가우시안을 먼저 거는 것이다. **엣지** 하나에 라플라시안을 걸면 잔물결 하나가 나오는데, 블롭은 가까이 붙은 엣지 두 개라 잔물결 두 개가 겹쳐 가운데에서 큰 극값을 만든다. 단, 라플라시안의 스케일이 블롭 크기와 맞아야(matched) 한다.

[특성 스케일 · 정규화 · DoG]

그래서 σ를 키워 가며 응답을 비교하면, **블롭**이 클수록 최대 응답을 주는 σ도 커져 최대 응답의 σ가 곧 블롭의 크기(**특성 스케일**, **characteristic scale**)가 된다. 그런데 σ가 커질수록 **가우시안**이 넓고 납작해져 라플라시안 응답 자체가 작아지므로, 그대로 비교하면 항상 작은 σ가 이긴다. 그래서 σ^2을 곱하는 **스케일 정규화**로 공정하게 비교한다. 여러 σ의 정규화 라플라시안을 **컨볼루션**하고, **스케일 공간**에서 응답의 극값을 찾는 것이 검출 절차다. 실제로는 σ가 조금 다른 두 가우시안 블러의 차인 **DoG**(**Difference of Gaussians**)가 정규화 **LoG**와 거의 같고, 피라미드를 만들며 어차피 해 두는 블러를 빼기만 하면 되어 훨씬 싸므로 DoG를 쓴다. **SIFT**가 바로 이 DoG 피라미드 위에서 극값을 찾는다. Harris가 '어디'만 답한다면 블롭 검출은 '어디 + 얼마나 큰지'를 답한다.

5-8

Q. 공변(covariant)과 불변(invariant)을 구분하고, 아핀 적응에 대해 설명하시오.

답

[공변 vs 불변]

공변은 영상에 변화를 가하고 피처를 뽑아낸 것이 피처를 뽑아낸 뒤 변화를 가한 것과 같다는 뜻으로, 결과가 영상을 따라 변한다. **블롭**의 위치와 스케일은 회전 · 확대에 공변이라 이미지를 2배 키우면 검출된 위치도 σ도 2배가 된다. 불변은 영상에 변화를 가해도 뽑아낸 값이 전혀 바뀌지 않는다는 뜻으로, 라플라시안 응답값이 그렇다. 검출기는 공변이어야 하고 **기술자**는 불변이어야 하는데, 위치는 따라 움직여야 같은 지점을 잡고, 그 지점을 설명하는 숫자는 같아야 짝을 지을 수 있기 때문이다.

[아핀 적응]

평면에 가까운 물체를 멀리서 볼 때 시점 변화는 **아핀 변환**으로 근사할 수 있다. 이때 **블롭**을 담은 창의 **2차 모멘트 행렬**을 타원으로 그리면 그 창의 특성 모양이 보이고, 같은 지점을 다른 시점에서 본 두 영역은 서로 아핀 관계라 그대로는 모양이 달라 매칭이 안 된다. 그래서 각 영역을 같은 크기의 원으로 펴서(normalization) 비교한다. 이것이 공변 검출에서 불변 기술로 넘어가는 다리이고 6주차 **기술자**의 전제다.

6-1

Q. 특징 기술자가 왜 필요하며, 좋은 특징의 조건은 무엇인지 설명하시오.

답

[필요성]

5주차에서 Harris와 **LoG**로 점을 찾았지만, 점의 좌표만으로는 두 사진의 점이 같은 곳인지 알 수 없다("We know how to detect points. Next question: How to match them?"). 그래서 점 주변의 생김새를 숫자 벡터 하나로 요약한 **기술자**(**descriptor**)를 만들어 두고, 두 점이 같은 곳인지를 벡터 사이의 거리로 판정한다. 지역 특징의 세 요소 중 관심점을 찾는 **Detection** 다음, 각 관심점 주변을 벡터로 뽑아 적는 **Description**이 이번 주이고, 이것이 두 시점의 대응을 정하는 **Matching**을 가능하게 한다.

[좋은 특징의 조건]

좋은 특징은 기하학적 · 광학적 변형을 겪어도 같은 특징이 다시 발견되는 **Repeatability**, 각 특징이 뚜렷하게 구별되는 **Saliency**, 특징의 수가 픽셀 수보다 훨씬 적은 **Compactness & efficiency**, 영상의 좁은 영역만 차지하는 **Locality**를 갖춰야 한다. 그리고 **기술자** 자체는 같은 점을 다른 조건에서 찍어도 벡터가 같은 불변(**Invariant**)이면서, 다른 점끼리는 벡터가 확실히 다른 변별력(**Distinctive**)이 있어야 한다. 검출기는 어디가 특징적인지 답하며 공변이어야 하고, 기술자는 그곳이 어떻게 생겼는지 답하며 불변이어야 한다.

6-2

Q. 템플릿 매칭의 한계와, 기술자로 쓸 수 있는 세 가지 방법(Image patch, Image gradients, Color histogram)의 문제를 설명하시오.

답

[템플릿 매칭]

가장 소박한 방법인 **템플릿 매칭**은 특징점 주변을 잘라 두고 다른 사진에서 매칭 점수가 가장 높은 곳을 찾는다. 하지만 잘라 둔 조각의 크기가 고정이고 픽셀을 격자 순서대로 비교하므로 Scaling, Rotation, Perspective에는 견디지 못하고, Illumination은 경우에 따라 다르다(depends). 차이를 그냥 제곱해 더하면 밝기가 바뀌는 즉시 무너지지만, 정규화 상관(**NCC**)을 쓰면 평균을 빼고 길이로 나눠 전체에 고르게 바뀐 밝기는 흡수하기 때문이다.

[세 가지 시도]

그래서 **기술자** 후보를 세 가지 시도한다. **Image patch**는 패치의 **화소값**을 그대로 쓰는 것으로, 기하와 외형이 변하지 않았다면 완벽하지만 절대 밝기 값에 민감하다. **Image gradients**는 픽셀 차이를 써서 절대 밝기에 불변이지만, 물체가 휘거나 늘어나는 변형(deformation)에 민감하다. **Color histogram**은 색의 개수만 세어 크기와 회전에 불변이지만, 위치 정보를 버려 다른 그림이 같은 히스토그램을 가질 수 있다. 결국 불변성을 키우면 변별력이 떨어지고 변별력을 키우면 불변성이 떨어지는 긴장이 있고, "**그래디언트**를 히스토그램으로 세되 위치를 완전히 버리지는 말자"는 것이 **SIFT**의 답이다.

6-3

Q. SIFT에 대해 설명하시오.

답

[개요]

SIFT(Scale **Invariant** Feature Transform, Lowe)는 영상의 내용을 이동 · 회전 · 크기 및 그 밖의 촬영 조건에 불변인 지역 특징 좌표로 옮겨 적는다. 단계는 Scale-space extrema detection과 Keypoint localization의 검출기 부분, Orientation assignment와 Keypoint **descriptor**의 **기술자** 부분으로 나뉜다.

[검출 — 스케일]

먼저 스케일에 대한 불변성을 위해, 같은 사진을 **가우시안**의 σ를 바꿔 가며 여러 번 블러하고 폭이 다른 두 가우시안을 뺀 **DoG**(**LoG**의 근사)를 만든다. σ를 키운다는 것은 더 넓은 범위를 평균 내 σ보다 작은 구조는 지우고 큰 구조만 남긴다는 뜻으로, 물체를 멀리서 본 것과 같다. σ가 두 배가 되면 이미지를 절반으로 줄여 다음 옥타브를 시작한다(4주차 **가우시안 피라미드**). 어떤 픽셀이 같은 스케일의 이웃 8개와 위아래 스케일의 9개씩, 총 **26개**보다 모두 크거나 작으면 키포인트 후보로 삼는다. 다음으로 극값 주변에 2차 함수를 맞춰 위치를 정밀하게 잡고, 대비가 낮은 점과 **엣지** 위의 점(라플라시안은 엣지에서도 강하게 반응한다)을 Harris의 **고윳값** 비율로 제거한다.

[기술 — 회전 · 128차원]

회전 불변성을 위해, 키포인트 스케일의 블러 영상에서 주변 **그래디언트** 방향을 36칸 히스토그램에 크기만큼 투표해 가장 큰 값을 기준 방향으로 삼는다. 최고 봉우리의 **80%** 이상인 봉우리가 있으면 그 방향으로도 키포인트를 하나 더 만든다(possibly multiple orientations). 마지막으로 기준 방향으로 돌린 16×16 영역을 4×4 셀로 나눠 셀마다 8방향 히스토그램을 만들면 **4×4×8 = 128**차원 **기술자**가 된다. 벡터를 길이 1로 정규화하고 큰 값을 잘라 조명 변화와 카메라 포화에도 불변하게 한다. 정리하면 검출은 공변, 기술은 불변이며, 평면 밖 회전 약 60도까지의 시점 변화와 낮 · 밤 수준의 조명 변화에 견딘다. 다만 상대적으로 느려 **SURF**와 **ORB**가 나온 동기가 되었다.

6-4

Q. SURF에 대해 SIFT와 비교하여 설명하시오.

답

[개요와 적분 영상]

SURF(Speeded-Up Robust Features)는 **SIFT**가 비교적 느려 더 빠른 버전이 필요해서 나왔으며, SIFT보다 3~7배 빠르면서 매칭 성능은 비슷하다. 핵심 아이디어는 **Haar** filter와 **Integral image**다. **적분 영상**(Integral image)은 각 칸에 원점부터 그 점까지 직사각형 안 **화소값**의 합을 담은 중간 표현으로, 한 번 만들어 두면 어떤 크기의 사각형이든 네 칸만 읽어 합을 구한다.

[검출]

검출에는 **헤시안 행렬**의 **행렬식**이 **국소 최대**인 곳을 관심점으로 삼는다. 두 방향이 함께 휘어 있을 때만 행렬식이 커져 **엣지**가 자동으로 걸러지고, 위치와 스케일을 모두 행렬식 하나로 고른다. 이때 **가우시안** 2차 미분을 **박스 필터**로 근사해 **적분 영상**으로 빠르게 계산한다. **SIFT**가 영상을 반복해서 줄였다면 **SURF**는 적분 영상 덕분에 영상은 그대로 두고 필터를

키워 가며(up-scaling of the filter at constant cost) 스케일을 훑는다. 극값 찾기와 위치 **보간**은 SIFT와 같다.

[방향과 기술자]

방향은 반지름 6s의 원 안에서 **Haar 웨이블릿** 응답 dx, dy를 벡터로 찍고, 60° 부채꼴 창을 돌려 가며 창 안 응답의 합 벡터가 가장 긴 방향을 주방향으로 삼는다. **기술자**는 20s 창을 4×4로 나눠 영역마다 Σdx, Σdy, Σ|dx|, Σ|dy|를 적어 **64차원**이고, 합과 절댓값 합을 함께 적어야 줄무늬처럼 부호가 번갈아 바뀌는 무늬와 균일한 면을 구별한다. 길이 1로 정규화해 대비에 불변하게 하고, 매칭 때는 라플라시안의 부호로 밝은 **블롭**과 어두운 블롭을 미리 갈라 속도를 높인다. 블러와 회전에는 강하지만 시점 변화와 조명 변화에는 약하다.

6-5

Q. HOG에 대해 설명하시오.

답

[목적]

HOG(Histograms of Oriented Gradients)는 사람 검출(human detection)을 위해 제안됐다. **SIFT** · **SURF**가 키포인트 하나하나에 벡터를 붙여 매칭했다면, HOG는 64×128 창 전체를 벡터 하나로 만들어 분류기(**SVM**)에 넣고 사람이 있는지를 판정한다. 같은 **그래디언트** 방향 히스토그램을 쓰지만 목적이 매칭이 아니라 분류라는 점이 다르다.

[4단계]

먼저 [−1 0 1] 필터로 **그래디언트**의 크기와 방향을 구하고(블러하면 가는 윤곽이 뭉개져 하지 않는다), 8×8 셀마다 0~180°를 20°씩 나눈 9칸 히스토그램에 크기만큼 투표한다. 칸 사이에 걸리면 가까운 정도에 따라 두 칸에 나눠 던져 각도가 조금 흔들려도 값이 뚝 바뀌지 않게 한다. 다음으로 2×2 셀을 묶은 블록 단위로 정규화(**block normalization**)하는데, 그래디언트 크기는 밝기 대비에 끌려다니므로 블록 안 값을 합이나 길이로 나눠 '그 동네에서 어느 방향이 상대적으로 우세한가'만 남긴다. 블록은 한 셀씩 겹치며 이동해 7×15 = 105개이고, 105 × 4셀 × 9 = **3780차원**의 벡터가 된다. 서 있는 사람은 늘 세로이므로 **SIFT**와 달리 회전은 맞추지 않는다.

6-6

Q. BRIEF와 BRISK에 대해 설명하시오.

답

[BRIEF]

BRIEF(Binary Robust Independent Elementary Features)는 값을 적지 않고 "어느 쪽이 더 밝은가"만 적는 이진 **기술자**다. 패치 안 두 점의 밝기를 비교해 뒤쪽이 더 밝으면 1, 아니면 0을 적는 **binary test**를 128 · 256 · 512번 해서 비트열을 만든다. 비교 전에 패치를 반드시 스무딩하는데, 화소 하나의 잡음이 대소 관계를 뒤집으면 비트가 통째로 바뀌기 때문이다. 비교 쌍은 미리 정해 둔 고정 패턴이어야 i번째 비트끼리 견줄 수 있으며, 중심 주변에서 **가우시안**으로 뽑는 방식이 가장 좋았다.

[해밍 거리 · 한계 · BRISK]

이진 **기술자**는 유클리디언 대신 **해밍 거리**로 비교한다. 두 비트열을 **XOR**하면 다른 자리만 1로 남고 그 개수가 거리이므로, 매칭이 N×N번의 거리 계산인 상황에서 매우 빠르다. 그래서 작고(compact) 계산이 쉽고 변별력이 높으며 매칭이 빠르다. 하지만 검출기가 없고, 비교 쌍이 고정 좌표라 영상이 돌거나 커지면 엉뚱한 화소를 비교하게 되어 평면 내 회전과 스케일 변화에 민감하다. **BRISK**(Binary Robust **Invariant** Scalable Keypoints)는 **SIFT** 같은 **스케일 공간** 검출과 **BRIEF** 같은 기술자를 결합해, 동심원 위 점 패턴에서 **가우시안**으로 평균 낸 값을 비교해 512비트 기술자를 만들고 해밍 거리로 비교한다.

6-7

Q. ORB에 대해 설명하시오.

답

[개요]

ORB(Oriented **FAST** and Rotated **BRIEF**)는 **SIFT**나 **SURF**를 대신할 효율적인 대안(an efficient alternative to SIFT or SURF)으로, 방향 개념이 없던 FAST 검출기에 빠르고 정확한 방향을 붙이고, 그 방향만큼 BRIEF의 비교 쌍을 돌리며, 돌려도 잘 작동하도록 비교 쌍을 학습으로 고른 방법이다.

[Oriented FAST · steered BRIEF]

흐름은 다음과 같다. **FAST**로 키포인트의 위치를 찾는다. FAST는 가운데 픽셀 주위 원 위의 16픽셀 중 가운데보다 확실히 밝거나 어두운 픽셀이 연속으로 n개(FAST-9, FAST-12) 이상이면 **코너**로 판정하며, 밝기 비교만 하므로 매우 빠르다. 그런데 FAST는 코너인지 아닌지만 말하고 **엣지**에서도 반응하므로 Harris 점수로 N best points를 고른다. 다음으로 패치 밝기의 무게중심(**Intensity Centroid**)을 구해, 패치 중심에서 무게중심으로 향하는 방향을 그 점의 방향으로 붙인다. 패치가 돌면 무게중심도 같이 돌아 방향이 회전을 따라간다. 그 방향의 회전 행렬을 **BRIEF** 비교 쌍에 곱해 패턴을 같이 돌린다(**steered BRIEF**).

[rBRIEF]

그런데 패턴을 방향에 맞춰 돌리면 비트들의 상관이 높아져 변별력이 떨어지므로, 새 쌍이 새 정보를 가져오도록(Uncorrelation) 하고 입력에 따라 0 · 1이 잘 갈리도록(High variance) 많은 키포인트로 비교 쌍을 시험해 greedy하게 256개를 고른다(**rBRIEF**). 결과는 **256비트 기술자**이고 **해밍 거리**로 매칭하며, **BRIEF**보다 회전에 강하다. 크기 변화는 영상 피라미드로 대응한다.

6-8

Q. SIFT, SURF, HOG, BRIEF, ORB를 비교하여 설명하시오.

답

[비교]

SIFT는 실수 **128차원**을 유클리디언 거리로 비교하며 스케일 · 회전 · 조명에 가장 안정적이지만 느리고 무겁다. **SURF**는 실수 **64차원**으로 SIFT보다 3~7배 빠르고 블러 · 회전에 강하지만 시점 · 조명 변화에 약하다. **HOG**는 64×128 창을 **3780차원**으로 적어 **SVM**이 처리하며, 사람처럼 형태가 정형화된 물체 검출에 쓰고 키포인트 매칭에는 쓰지 않는다.

BRIEF는 이진 **256비트**를 **해밍 거리**로 비교해 메모리가 작고 매칭이 매우 빠르지만 회전 · 스케일에 약하다. **ORB**는 BRIEF의 속도에 회전 강인성을 더해 실시간 · 모바일에 쓰이지만 정밀도는 SIFT보다 낮다. 실수 **기술자** 쪽으로 갈수록 정확하고 무겁고, 이진 기술자 쪽으로 갈수록 거칠고 가볍다. 실수 벡터는 유클리디언, 이진 벡터는 해밍 거리로 매칭한다.

7-1

Q. 피팅이 무엇이며, 최소제곱 피팅과 그 한계를 설명하시오.

답

[피팅]

지역 특징의 세 요소 중 마지막인 매칭은 두 시점의 **기술자** 사이에서 대응(correspondence)을 정하는 것이다. 그런데 기술자 거리만 재서 짝지으면 틀린 짝이 반드시 섞이므로, 짝들이 공통으로 따르는 하나의 변환을 찾고 그 변환에 맞지 않는 짝은 버리는데, 이것이 **피팅**이다. 입력인 점들에는 측정 위치의 잡음(noise), 엉뚱한 점(**이상치** · clutter)이나 여러 개의 직선, 가려져서 빠진 점(occlusion)의 문제가 있다. 어떤 점이 직선에 속하는지 안다면 **최소제곱**, 이상치가 있으면 강건한 피팅 · **RANSAC**, 직선이 여러 개면 투표(RANSAC, **허프 변환**), 직선인지조차 모르면 모델 선택을 쓴다.

[최소제곱과 한계]

최소제곱은 각 점에서 직선까지의 세로(y) 차이를 제곱해 더한 것을 최소화하며, 미분해 0으로 두면 **정규방정식**이 나온다. 하지만 좌표축을 돌리면 다른 직선이 나와 회전 불변이 아니고, 수직선에서는 완전히 실패한다. 그래서 직선까지의 진짜 수직 거리를 재는 **전체 최소제곱**(**TLS**)을 쓰면, 직선이 반드시 점들의 무게중심을 지나고 해는 점 분포 행렬의 가장 작은 고유값에 대응하는 고유벡터(법선)가 된다. 두 방법 모두 제곱 오차가 **이상치**에 엄청난 벌점을 주기 때문에 (squared error heavily penalizes outliers), 점 하나만 멀리 떨어져 있어도 직선이 그쪽으로 크게 기울어 나머지 점 전부를 놓친다.

7-2

Q. RANSAC에 대해 설명하시오. (단계, 파라미터, 장단점)

답

[단계]

RANSAC(RANdom SAmple Consensus)은 **이상치**가 있는 상황에서 모델을 맞추는 매우 일반적인 틀이다. 작은 부분집합을 무작위로 골라 모델을 맞추고, 나머지 점 중 모델에 가까운 점(**인라이어**)을 모두 찾고 나머지는 이상치로 버리며, 이를 여러 번 해서 가장 좋은 모델을 고른다. 직선 예제로 보면 ① 직선은 점 2개로 정해지므로 최소 부분집합 2개를 뽑고 (Randomly select minimal subset), ② 두 점을 지나는 직선을 가정하고(Hypothesize a model), ③ 모든 점에서 그 직선까지의 거리를 재고(Compute error function), ④ 거리가 임계값 이내인 점을 인라이어로 고르며(Select points consistent with model), ⑤ 다른 두 점으로 이 가설-검증 루프를 반복한다. 마지막에 인라이어가 가장 많은 모델을 골라 그 인라이어 전부로 **최소제곱**을 다시 돌려 다듬는데, 이상치가 빠졌으니 이번에는 최소제곱이 제 역할을 한다.

[파라미터와 장단점]

파라미터는 네 개다. 초기 점 수 s는 모델을 정하는 최소 개수(직선 2, 어파인 3, **호모그래피** 4)로, 적게 뽑을수록 모두 **인라이어**일 확률이 크다. 거리 임계값 t는 인라이어일 확률이 0.95 정도가 되도록 잡음의 크기에 맞춘다. 표본 횟수 N은 확률 0.99 정도로 적어도 한 번은 **이상치** 없는 표본을 뽑도록 정하며, 이상치 비율이 커질수록 폭발적으로 늘지만 점의 총

개수와는 무관하다. **합의 집합** 크기 d는 모델을 받아들이는 최소 인라이어 수다. 장점은 단순하고 일반적이며 다양한 문제에 적용되고 실제로 잘 작동한다는 것이고, 단점은 조정할 파라미터가 많고, 인라이어 비율이 낮으면 반복이 너무 많아지거나 실패하며, 최소 표본만으로는 좋은 초기값을 항상 얻지 못한다는 것이다.

7-3

Q. 매처의 종류와 영상 정렬(image alignment)에 대해 설명하시오.

답

[매처]

Brute-Force 매처는 첫 집합의 **기술자** 하나를 꺼내 둘째 집합의 모든 특징과 대 보고 가장 가까운 것을 돌려줘 정확하지만 느리고, **FLANN**(Fast Library for Approximate Nearest Neighbors) 매처는 대규모 · 고차원 특징에서 근사 **최근접 이웃**을 빠르게 찾는다. 거리는 실수 기술자(**SIFT**)는 유클리디언, 이진 기술자(**ORB**)는 해밍을 쓴다. 정답 대응을 모를 때는 특징을 추출하고, 잠재적 매칭을 계산한 뒤, 변환을 가정(Hypothesize)하고 그 변환과 일치하는 다른 매칭을 찾아 검증(Verify)하는 루프를 도는데, 이것이 **RANSAC**의 가정-검증 루프다.

[영상 정렬]

영상 정렬은 두 사진이 같은 장면을 본다면 한쪽을 변환해 다른 쪽에 맞추는 것으로, 겹침이 작거나(small degree of overlap), 가림 · 복잡한 배경이 있거나, 밝기가 바뀌는 어려움이 있다. 대부분의 픽셀이 일치하는 정렬을 찾는 직접(픽셀 기반) 정렬과, 추출한 특징들이 일치하는 정렬을 찾는 **특징 기반 정렬**이 있다. 정렬은 곧 **피팅**으로, 한 영상의 특징에 모델을 맞추던 것을 두 영상의 특징 쌍 사이의 변환에 맞추는 것으로 바꾼 것이다. 어파인 변환은 미지수 6개의 선형 시스템이라 짝 하나가 식 두 개를 주어 최소 세 쌍이 필요하고, 더 많으면 **최소제곱**으로 푼다. 하지만 카메라를 돌려 찍은 사진을 **평행이동**만으로 이어 붙이면 이음새마다 어긋나므로(Translation-only alignment is not enough) **호모그래피**를 써야 한다.

7-4

Q. 호모그래피가 무엇이며, 어떤 경우에 쓸 수 있고 어떻게 구하는지 설명하시오.

답

[정의]

두 카메라 중심에서 같은 장면을 볼 때 한 투영면을 다른 투영면으로 옮기려면 변환의 위계(**translation**, **Euclidean**, **similarity**, **affine**, **projective**) 중 가장 일반적인 사영 변환이 필요하고, 이것이 **호모그래피** = 평면 사영 변환(plane projective transformation)이다. 3×3 행렬이지만 전체에 상수를 곱해도 같은 변환이라 **자유도**는 8이고, 대응점 4쌍이 필요하다. 직선은 직선으로 가지만 평행성은 깨진다. 같은 사진을 겹칠 때 **평행이동**이나 어파인으로는 맞지 않던 벽과 책상이 호모그래피로만 정확히 맞는다.

[쓰는 경우와 응용]

호모그래피는 장면이 평면일 때(벽화를 두 시점에서), 장면이 아주 멀어 근사적으로 평면일 때(요세미티 원경), 그리고 카메라 중심이 같을 때(제자리에서 회전만) 쓸 수 있다. 세 번째는 장면이 평면이 아니어도 되는데, 카메라가 이동하면 가까운 물체와 먼 물체가 다르게 움직이는 **시차**가 생겨 깨지지만 제자리에서 돌기만 하면 모든 광선이 같은 중심을 지나기 때문이다. 그래서 **파노라마**는 제자리에서 돌며 찍는다. 이를 이용해 거리 예술, 기울어진 바닥 무늬를 정면으로 펴는 rectification, 가상 카메라 회전, 정면화 후 이어 붙이기를 할 수 있다.

[DLT]

구하는 방법(**DLT**)은 점을 **동차 좌표**로 바꾸고, H를 곱하고, 세 번째 성분으로 나눠 되돌리는 관계에서 출발한다. 대응점 하나마다 알 수 없는 배율을 세 번째 식으로 소거하면 h에 대해 선형인 식 두 개, 즉 2×9 행렬이 나온다. 이를 n개 쌓아 2n×9 행렬로 만들고, h = 0이라는 자명한 해를 피하려고 길이를 1로 고정한 뒤 **SVD**를 계산해 **가장 작은 특이값**에 대응하는 벡터를 h로 저장하고 3×3으로 reshape한다. 나눗셈 때문에 **호모그래피**는 선형이 아니고, 이 방법은 **이상치**에 약하므로 **RANSAC**과 함께 쓴다.

7-5

Q. RANSAC으로 호모그래피를 추정하는 과정과 비율 검사에 대해 설명하시오.

답

[동시 추정]

영상 대응 파이프라인은 Harris 등으로 특징점을 검출하고, 특징을 기술한 뒤, 특징 매칭과 **호모그래피** 추정을 **RANSAC**으로 동시에 한다. 선형 **최소제곱**은 변환이 선형일 때만 제대로 작동하는데 호모그래피는 그렇지 않고, **이상치**도 잘 다루지 못한다. **SIFT**로 찾은 짝에는 나쁜 대응이 섞이고, 최소제곱은 좋은 짝과 나쁜 짝을 모두 평균 내 오염된 '**평균 변환**'을 찾는다. **평행이동** 예로 보면 짝 하나를 골라 그 이동에 동의하는 짝을 세는 것을 반복해, **인라이어**가 가장 많은 모델을 고른다. 좋은 짝들은 같은 변환을 따라 서로 동의하고 나쁜 짝은 제각각이라 다수결에서 밀려난다. 호모그래피라면 4쌍을 뽑아 H를 구하고 인라이어를 세는 루프를 돌아, 가장 많은 인라이어의 H를 그 인라이어 전부로 다시 추정한다. 그래서 H와 맞는 매칭 목록이 함께 나온다.

[비율 검사]

믿을 수 없는 매칭은 **비율 검사**로 미리 버린다. 가장 가까운 이웃까지의 거리를 두 번째로 가까운 이웃까지의 거리와 비교하는데, 변별력이 없는(not distinctive) 특징이면 이 비율이 1에 가깝게 높다. 확실한 짝이 있으면 1순위가 매우 가깝고 2순위는 확연히 멀어 비율이 낮지만, 반복 패턴은 비슷한 후보가 밀집해 1 · 2순위가 거의 같고, 짝이 아예 없는 점은 둘 다 엉뚱해 비슷하게 멀다. 그래서 비율이 0.8 정도보다 작을 때만 인정하면 오매칭을 크게 줄이고, **RANSAC**에 들어가는 **이상치** 비율이 낮아져 필요한 반복 횟수도 크게 줄어든다.

7-6

Q. 이미지 스티칭(파노라마)과 블렌딩에 대해 설명하시오.

답

[스티칭]

두 장을 이어 붙이려면 **SIFT** 특징을 매칭하고, **RANSAC**으로 **이상치**를 버린 뒤, 구한 H로 한 장을 **와핑**해 이어 붙인다. 여러 장이라면 영상 하나를 기준으로 고르고 나머지를 보통 한 장씩 그쪽으로 와핑해 섞는다. 순서는 같은 위치에서 카메라를 광학 중심 기준으로 회전하며 연속으로 찍고, 둘째와 첫째 영상 사이의 변환을 계산해 둘째를 옮겨 겹치고, 섞어 모자이크를 만든 뒤 영상이 더 있으면 반복하는 것이다. 같은 위치에서 회전만 하는 조건을 지켜야 장면에 깊이가 있어도 **호모그래피** 하나로 정확히 겹친다.

[블렌딩]

그런데 여러 장을 이어 붙이면 경계가 눈에 띈다. 원인은 센서에 들어오는 빛의 각도 의존성 때문에 사진 가장자리가 어두워지는 **비네팅**(**vignetting**)과 작은 어긋남(small misalignments)이다. 이를 지우는 **블렌딩**으로, **페더링**(**Feathering**)은 각 픽셀에 투명도 α를 붙여 겹치는 구간에서 왼쪽은 1→0, 오른쪽은 0→1로 서서히 바꿔 더한다. 간단하지만 섞는 폭이 넓으면 어긋난 곳이 겹쳐 보이고 좁으면 이음새가 남는다. **다중 대역 블렌딩**(**Multi-band blending**)은 4주차의 영상 피라미드를 쓰는 다중 해상도 기법으로, 두 영상을 **라플라시안 피라미드**로, 마스크를 **가우시안 피라미드**로 분해해 저주파는 넓게, 고주파는 좁게 층마다 따로 섞은 뒤 복원한다. 그래서 이음새는 감추면서 날카로운 디테일은 보존된다.

MIX

주차를 엮는 문제

한 주제를 여러 주차의 교안으로 이어 쓰기

X-1 2 · 3 · 4 · 5 · 6 · 7주차

Q. 두 장의 사진으로 파노라마를 만드는 과정을 지금까지 배운 내용과 엮어 설명하시오.

답

[촬영 · 전처리 · 스케일]

카메라는 3차원 장면을 2차원으로 투영해 이미지를 만들며(2주차), 두 사진을 **호모그래피** 하나로 이으려면 카메라 중심을 고정하고 회전만 하며 찍어야 한다. 촬영된 **디지털 이미지**에는 **노이즈**가 있고 미분은 노이즈를 키우므로 **가우시안 필터**로 먼저 부드럽게 한다(3주차). 같은 물체가 사진마다 다른 크기로 찍힐 수 있으므로, 블러 후 축소를 반복해 **에일리어싱** 없이 **가우시안 피라미드**, 즉 **스케일 공간**을 만든다(4주차).

[검출 · 기술 · 매칭 · 합성]

이 **스케일 공간**에서 이웃한 블러 영상의 차인 **DoG**로 위치와 크기를 함께 가진 **블롭**을 검출하고, **엣지** 위의 불안정한 점은 Harris의 **고윳값** 원리로 버린다(5주차). 검출된 점마다 **SIFT**로 기준 방향과 크기에 맞춘 패치를 **4×4×8 = 128**차원 **그래디언트** 히스토그램으로 기술하는데, 검출은 공변이고 기술은 불변이라 두 사진에서 같은 지점이 같은 벡터를 가진다(6주차). 두 사진의 **기술자**를 거리로 매칭하고 **비율 검사**로 모호한 짝을 버린 뒤, **RANSAC**으로 4쌍씩 뽑아 **DLT**로 **호모그래피**를 세우고 가장 많은 짝이 동의하는 H를 고른다. H로 한 장을 **와핑**하면서 소수점 좌표는 **보간**으로 값을 가져오고(4주차), **비네팅**과 작은 어긋남으로 생긴 이음새는 **라플라시안 피라미드** 기반 **다중 대역 블렌딩**으로 지운다(7주차).

X-2 4 · 5 · 6주차

Q. 크기(스케일) 변화에 대응하는 방법들을 엮어 설명하시오.

답

[피라미드와 블롭]

검출기의 창은 크기가 고정이라, 같은 물체가 확대되면 Harris는 뾰족한 모서리를 완만한 곡선으로 보고 **엣지**로 분류해 버린다. 4주차의 **가우시안 피라미드**는 창을 키우는 대신 이미지를 블러 후 절반씩 줄여 같은 창으로 모든 층을 훑게 해, 큰 물체는 위층에서, 작은 물체는 아래층에서 잡는다. 5주차의 **블롭** 검출은 σ를 키워 가며 **LoG** 응답을 비교해 최대 응답의 σ를 블롭의 크기(**특성 스케일**)로 삼고, σ가 커질수록 응답이 작아지는 문제를 σ^2을 곱하는 **스케일 정규화**로 보정한다.

[SIFT · SURF · ORB]

6주차의 **SIFT**는 이 둘을 합쳐 σ를 바꾼 블러와 옥타브마다 절반 축소로 **스케일 공간**을 만들고, **LoG** 대신 싼 **DoG**에서 위아래 스케일까지 **26개** 이웃과 비교해 극값을 찾은 뒤 그 σ에 맞춘 패치로 기술한다. **SURF**는 영상은 그대로 두고 **적분 영상** 덕분에 **박스 필터**를 키워 가며 스케일을 훑고, **ORB**는 스케일 개념이 없는 **FAST**와 **BRIEF**를 영상 피라미드의 층마다 적용한다. 반면 패치 크기가 고정인 BRIEF는 스케일 변화에 약하다. 결국 모든 방법이 영상을 줄이든 필터를 키우든 물체에 맞는 크기를 찾아 그 크기의 영역을 본다는 같은 생각이다.

PAIR

헷갈리는 짝 비교표

"A와 B를 비교하시오" 유형 대비 — 주차를 함께 적었다

짝	주차	A	B
Radiometry vs Photometry	2	기계가 재는 물리적인 빛의 방향 · 양 (적외선 · 자외선 포함)	사람이 눈으로 느끼는 빛의 세기
필터링 vs 워핑	2 · 3	화소값(range)을 바꾼다 — 블러 · 엣지	좌표(domain)를 바꾼다 — 회전 · 원근
아핀 vs 호모그래피	2 · 7	6자유도 · 3쌍 · 평행성 보존	8자유도 · 4쌍 · 직선만 보존(원근)
내부 vs 외부 파라미터	2	초점거리 · 주점 · 픽셀 배율 · skew (카메라 고유)	회전 · 이동 (카메라 위치 · 자세)
원근 왜곡 vs 방사 왜곡	2	카메라 기울기 때문에 수직선이 모임 → 시프트 렌즈 · 호모그래피	렌즈 결함 · 가장자리 휨(배럴 · 핀쿠션) → 왜곡 계수
컨볼루션 vs 교차 상관	3	필터를 뒤집어 겹침 — 필터링	뒤집지 않음 — 유사도 · 템플릿 매칭
평균 vs 가우시안 필터	3	같은 가중치 → 네모 자국	중심 가중 · 회전 대칭 · σ로 조절
가우시안 vs 미디언 필터	3	선형 · 가우시안 잡음에 적합 · 이상치를 번지게 함	비선형 · 소금 · 후추 잡음에 적합 · 정렬 비용
DoG (3주차) vs DoG (5 · 6주차)	3 · 5 · 6	Derivative of Gaussian — 가우시안의 1차 미분, 엣지	Difference of Gaussians — 블러 두 장의 차, LoG 근사 · 블롭 · SIFT
가우시안 vs 라플라시안 피라미드	4	블러 + 축소 반복 — 저주파 · 복원 불가 · 검출에 활용	층 사이 차이(잔차) — 고주파 · 정확 복원 · 블렌딩 · 압축
라플라시안 피라미드 vs 웨이블릿	4	계수 약 4/3배 · 방향 구분 없음	계수 원본과 같음 · 가로 · 세로 · 대각 디테일
바이리니어 vs 바이큐빅	4	4점 · 직선 · 흐림 · 빠름	16점 · 곡선 · 선명 · 느림 · 오버슈트
1차 미분 vs 2차 미분 엣지	3 · 5	극값 = 엣지 (Sobel · Canny)	영교차 = 엣지 · 이중선 · 잡음에 더 민감 (LoG)
Harris vs 블롭(LoG)	5	'어디'만 — 스케일에 약함	'어디 + 얼마나 큰지' — 특성 스케일
Harris vs Shi-Tomasi	5	행렬식과 대각합으로 만든 점수	작은 고윳값 자체를 점수로
공변 vs 불변	5 · 6	결과가 영상을 따라 변함 — 검출기	결과가 그대로 — 기술자
검출기 vs 기술자	5 · 6	어디가 특징적인가 → 좌표 · 스케일 · 방향	어떻게 생겼나 → 고정 길이 벡터

SIFT vs SURF	6	DoG · 영상을 줄임 · 36칸 방향 · 128차원	헤시안 행렬식 · 필터를 키움 · 60° 창 · 64차원 · 3~7배 빠름
SIFT vs HOG	6	키포인트 패치 · 회전 맞춤 · 매칭	창 전체 · 회전 안 맞춤 · 블록 정규화 · 분류(SVM)
BRIEF vs ORB	6	검출기 없음 · 회전 · 크기에 약함	FAST + 방향 + 피라미드 · 비교 쌍 회전 · rBRIEF 학습
유클리드 vs 해밍 거리	6 · 7	실수 벡터 (SIFT · SURF)	이진 벡터 (BRIEF · ORB) · XOR로 빠름
세로 최소제곱 vs TLS	7	세로 차이 · 회전 불변 아님 · 수직선 불가	수직 거리 · 무게중심 통과 · 작은 고윳값 방향이 법선
Brute-Force vs FLANN	7	모든 쌍 비교 · 정확 · 느림	근사 최근접 · 대규모에서 빠름
직접 정렬 vs 특징 기반 정렬	7	모든 화소 비교 · 무거움 · 초기값에 민감	특징 몇백 개 · 빠름 · 큰 변형에 강함
페더링 vs 다중 대역 블렌딩	7	폭 하나의 투명도 그러데이션 — 유령 또는 이음새	피라미드 층마다 다른 폭 — 이음새 없이 디테일 보존

출처 — 컴퓨터비전 1~7주차 교안(2026-2학기, 최재영 교수)과 이 사이트의 주차별 정리 페이지
‘직접 정리’ 표시가 붙은 답은 직접 적은 답이고, 연한 초록 테두리 문단은 그 답에 교안 내용을 이어 붙인 부분이다.